计算机组成原理

2.1 数值型数据的表示

第2章 数据表示、运算与校验

主要介绍：

①数字型数据的计数制、符号数的表示、定点数和浮点数；

②基本的运算方法；

③字符的表示；

④常用的数据校验方法；

计算机处理的对象——数据

数据的机器码表示---
利用逻辑电路进行处理

计算机数据机器表示需要考虑的因素

- 支持的数据类型：数值数据（小数、整数、实数和复数）、非数值数据（文字、图片等）
- 数值范围, 数值精度
- 数值存储和处理所需的硬件代价
  - 少量简单的基本符号表示大量复杂的信息
  - 状态简单,电路实现简单
  - 运算处理方便
- 是否有利于软件移植

2.1 数值数据表示方法

- 计算机数值数据表示要素
- 进位制数
- 带符号数的表示：机器数
- 定点、浮点表示法：小数点的表示

真值和机器数

机器数：用0和1编码的计算机内部的0/1序列

真值：机器数真正的值，即：现实中带正负号的数

3.1415926

-3.1415926

数值数据表示三要素：

(1) 进制转换

(2) 正负号的表示

(3) 小数点的表示

2.1 数值型数据的表示方法

2.1.1 进位计数制

※ 数制的基与权

在任一数制中，每一个数位上允许使用的记数符号的个数被称为该数制的基数。

每1位都对应1个表示该位在数码中的位置的值，这个值就称为数位的权值w。

[例] $1\underline{2}8_{10}$，$1\underline{1}01_2$

$W=10^2$　　$W=2^3$

1. 常用的几种进位制

（1）2进制：0、1

（2）8进制：0、1、2、…、7

（3）16进制：0、…、9、A、B、C、D、E、F

2. 进制之间的转换

（1）整数 10 → 2 (除2取余法)

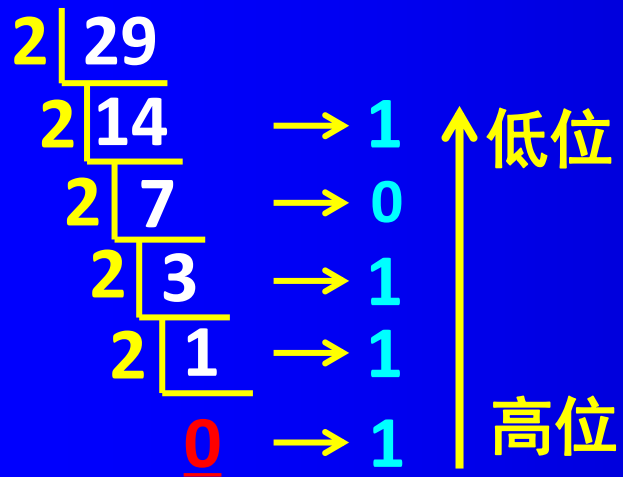
（2）小数 10 → 2 (乘2取整法)

（3）整数 2 → 10(按权相加)

（4）小数 2 → 10(按权相加)

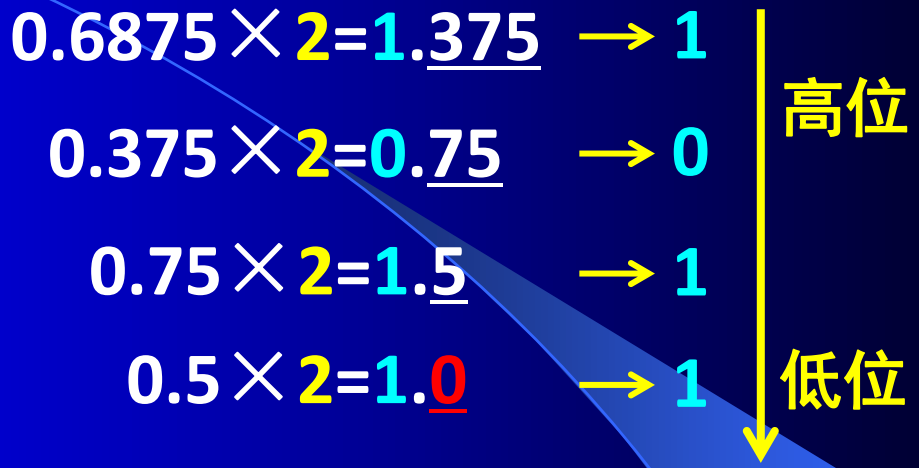
（5）16进制↔2进制(逐位转换/分组转换)

(1) $29_{10} \rightarrow X_2$ 除2取余



$X_2=11101_2$

(2) $0.6875_{10} \rightarrow X_2$ 乘2取整



$X_2=0.1011_2$

(3) $1101.11_2 \rightarrow X_{10}$ 按权相加

$X_{10}=1\times2^3+1\times2^2+0\times2^1+1\times2^0+1\times2^{-1}+1\times2^{-2}$

$=13.75_{10}$

(4) $111101_2 \rightarrow X_{16}$ 4位分组、按组转换

$X_{16}=111101=0011\ 1101=\underline{0011}\ \underline{1101}=3D_{16}$

(5) $28AF_{16} \rightarrow X_2$ 逐位转换

$X_2=28AF$

$=\underline{0010}\ \underline{1000}\ \underline{1010}\ \underline{1111}$

(6) $28AF_{16} \rightarrow X_{10}$ 按权相加

$X_{10}=28AF=2\times16^3+8\times16^2+10\times16^1+15\times16^0$

$=10415_{10}$

练习:

1.将（116）$_{10}$转换为二进制数

A 1110100　B 1110101　C 0110100　D1111100

2.将（0.625）$_{10}$转换为二进制数

A 0.101　B 0.111　C 0.100　D 0.001

3.将（**1101.11**）$_{2}$转换为十进制数

A 13.75　B 13.65　C 10.65　D 10.75

练习答案:

1.将（116）$_{10}$转换为二进制数

A 1110100　B 1110101　C 0110100　D1111100

2.将（0.625）$_{10}$转换为二进制数

A 0.101　B 0.111　C 0.100　D 0.001

3.将（1101.11）$_{2}$转换为十进制数

A 13.75　B 13.65　C 10.65　D 10.75

2.1.2 带符号数的表示

- ✓ 真值 (书写用)
  - 将用+ -表示正负的二进制数称为符号数的真值
- 机器不能识别书写格式，计算机如何表示负数？
- ✓ 机器码 (机器内部使用)
  - 将符号和数值一起编码表示的二进制数称为机器码
- ✓ 四种机器码
  - 原码 Signed magnitude ： 用来表示浮点数的尾数
  - 反码 One’s complement： 已不用于表示数值数据
  - 补码 Two’s complement ： 50年代开始成为整数标准
  - 移码 Biased notation： 用于表示浮点数阶码

2.1.2 带符号数的表示

数的符号表示规则：

“0”表示正号“+”，“1”表示负号“–”

二进制数的码制：原码、反码、补码和移码

1、原码

一个二进制数，用0-1代码表示符号，数值位不变就得到与该二进制数真值对应的原码

真值	+1001010	-1001010
	↓↓↓↓↓↓↓	↓↓↓↓↓↓↓
原码	01001010	11001010

2.1.2 带符号数的表示

1、原码

原码表示法的思想：

让数码序列的最高位为符号位，符号位为0表示该数为正，为1表示该数为负；数码序列的其余部分为有效数值，用二进制的绝对值表示。

真值	+1001010	-1001010
	↓↓↓↓↓↓↓	↓↓↓↓↓↓↓
原码	01001010	11001010

原码符号位是人为规定的

练习

- $[+0]_{原}=$
- $[-0]_{原}=$
- $[-0.1111]_{原}=$
- $[\ 0.1111]_{原}=$
- $[\ 1110]_{原}=$
- $[-1110]_{原}=$

最高位为符号位，0：正，1：负，数值位不变

练习

符号位（+0，-1）+绝对值

- $[+0]_{原}=0.000\ldots0$
- $[-0]_{原}=1.000\ldots0$
- $[-0.1111]_{原} = 1.1111$
- $[\ 0.1111]_{原} = 0.1111$
- $[\ 1110]_{原} = 01110$
- $[-1110]_{原} = 11110$

原码定义

①若真值为纯小数，它的原码形式为$X_0.X_1X_2\cdots X_n$，其中X_0为符号位，则

$$[X]_{原}=\begin{cases} X & 0\leqslant X<1 \\ 1-X=1+|X| & -1<X\leqslant 0 \end{cases}$$

表示纯小数的范围：$(-1,\ 1)$

$[-(1-2^{-n}),\ 1-2^{-n}]$

0的表示：$[+0]_{原}=0.00\ldots0$

$[-0]_{原}=1.00\ldots0$

若x=+0.1011, 则5位字长的X原=

8位字长的X原=

若x=+0.1011, 则5位字长的X原=

8位字长的X原=

若x=+0.1011, 则5位字长的X原=0.1011

8位字长的X原=0.1011(000)

若x=+0.1011, 则5位字长的X原=1.1011

8位字长的X原=1.1011(000)

原码定义

②若纯整数的原码序列为$X_0X_1\ldots Xn$，其中X_0表示符号位，则

$$[X]_{原}=\begin{cases} X & 0\le X<2^n \\ 2^n-X=2^n+|X| & -2^n<X\le 0 \end{cases}$$

表示纯整数的范围：$(-2^n,\ 2^n)$

$[(2^n-1),\ (2^n-1)]$

0的表示：$[+0]_{原}=000\ldots0$

$[-0]_{原}=100\ldots0$

字长为8位的原码

表示范围为：-127～+127

$[+127]_{原}$ =0 1111111

$[-127]_{原}$ =1 1111111

数值“0”有两种原码形式：

$[+0]_{原}$=0 0000000

$[-0]_{原}$=1 0000000

若x=+1011, 则5位字长的X原=

8位字长的X原=

若x=+1011, 则5位字长的X原=

8位字长的X原=

若x=+1011, 则5位字长的X原=01011

8位字长的X原=0(000)1011

若x=+1011, 则5位字长的X原=11011

8位字长的X原=1(000)1011

原码表示区间

- 纯小数$x_0.x_1x_2\cdots x_n$
  - $-0.11\cdots1 \sim +0.11\cdots1$
  - $[-(1-2^{-n}), 1-2^{-n}]$ 或 $(-1,1)$
- 纯整数$x_0x_1x_2\cdots x_n$
  - $-011\cdots1 \sim +011\cdots1$
  - $[-(2^n-1), 2^n-1]$ 或 $(-2^n, 2^n)$
- 对称区间

原码特点

表 2.1　原码的表示方法

	真　　值	原　　码
正数	$x=+0.x_1x_2\cdots x_n$	$[x]_{原}=0.x_1x_2\cdots x_n$
负数	$x=-0.x_1x_2\cdots x_n$	$[x]_{原}=1.x_1x_2\cdots x_n$
正数	$x=+x_1x_2\cdots x_n$	$[x]_{原}=0x_1x_2\cdots x_n$
负数	$x=-x_1x_2\cdots x_n$	$[x]_{原}=1x_1x_2\cdots x_n$

- ✓ 直观易懂：第一位为符号位，其他为数值位
- ✓ 0的表示不唯一
- ✓ 加、减运算方式不统一：
  加法：符号同数值部分直接加，符号异数值部分减。
  减法：先比较两个运算数绝对值大小，符号位同绝对值大的那个数的符号位。

2、反码

$$[X]_{反}=\begin{cases} X & 0\le X<2^n \\ 2^{n+1}-1+X & -2^n<X\le 0 \end{cases}$$

$$[X]_{反}=\begin{cases} X & 0\le X<1 \\ 2-2^{-n}+X & -1<X\le 0 \end{cases}$$

- 反码，符号位表示方法与原码相同，**正数的反码与正数的原码形式相同,负数的反码，符号位为1，数值部分取反。**
- **取值范围同原码**

① 正数情况

$X_{反}=X_{原}$（$X\geq 0$）

[例] X= +1101001 (真值+105)

$X_{反}=X_{原}=\underline{0}$ 1101001

② 负数情况

符号位保持为“1”，数值位分别“按位取反”

[例] X= -1101001 (真值-105)

$X_{原}=\underline{1}$ 1101001

$X_{反}=\underline{1}$ 0010110

字长8位的反码

表示范围为：-127 ～ +127

$[+127]_{反}=0\ 1111111$

$[-127]_{反}=1\ 0000000$

数值“0”也有两反码形式：

$[+0]_{反}=0\ 0000000$

$[-0]_{反}=1\ 1111111$

反码特性

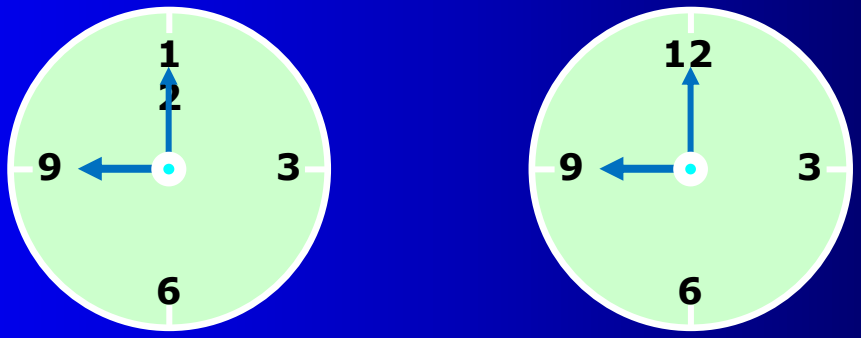
- 两个零
- 求反用逻辑门容易实现
- 运算仍然很复杂

思考

- 零是否可以唯一表示?
- 运算简单?

有趣的时钟

- 9与-3等效

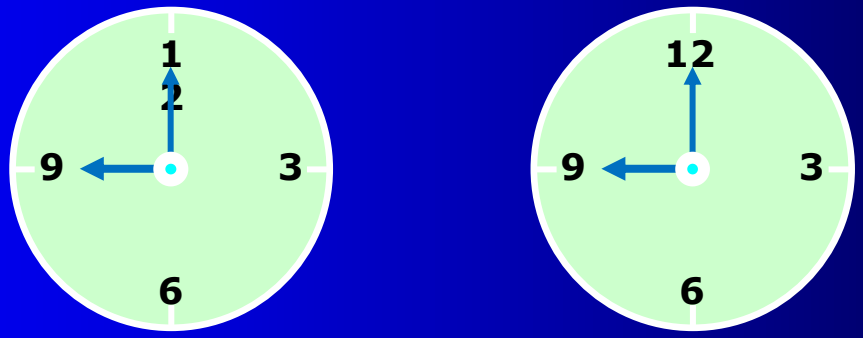


模的概念

- 模（或称模数）是一个数值计量系统的计量范围，记作mod或者M。
- 时钟：模12
- 超出计量范围内的自动舍去模数。比如14点也是下午2点。

有趣的时钟

- 9与-3等效



9和-3对模12互补

同理：-8和4互补；-5的补码是7

模的特点：

- 负数的补码可以用模数加上该负数获得，即用补码表示负数的时候可以利用模的性质转换成正数
- 小于模的正数相对于模而言是其本身
- 大于模的正数相对于模而言减去模的倍数的余数

结论：引入模的概念可将原码运算中的减法变成加法运算

3、补码

①补码的统一定义式　$[X]_{补} = M + X \pmod{M}$，其中X可正可负

关键在于求M

正数则模舍去，负数则模减去真值的绝对值。

②纯小数的补码定义公式

若纯小数的补码序列为$X_0.X_1X_2\cdots X_n$，其中X_0为符号位，模为2，则

$$[X]_{补}=\begin{cases} X & 0\leq X<1 \\ 2+X=2-|X| & -1\leq X<0 \end{cases} \pmod 2$$

表示纯小数的范围：

[-1，1）

[-1，$(1-2^{-n})$]

0.111...11

0的表示：

$[0]_{补}=0.00...0$

例题

例 若X=0.1011，则8位字长的 $X_{补}$= 0.1011000

(2+0.1011000) mod 2 = 0.1011000

$X_{原}$ = 0.1011000

例 若X＝－0.1011，则5位字长$X_{补}$＝(2 - 0.1011)mod2=1.0101

写成8位字长，则$X_{补}$＝ （ 2－0.1011000）mod 2＝

1.0101000

$X_{原}$＝ 1.1011000

符号位可以直接参与运算

③纯整数的补码定义式

若纯整数的补码序列为$X_nX_{n-1}\cdots X_1X_0$，即连同符号位有n + 1位，以2^{n+1}为模有

$$[X]_{补}=\begin{cases} X & 0\le X<2^n \\ 2^{n+1}+X=2^{n+1}-|X| & -2^n\le X<0 \end{cases} \pmod{2^{n+1}}$$

表示纯整数的范围：$[-2^n,\ 2^n)$

111...11

$[-2^n,\ (2^n-1)]$

0的表示：$[0]_{补}=000\ldots0$

补码表示法

例 若X = 1011，则5位字长的$X_{补}$=01011

8位字长的$X_{补}$=00001011

(1011) mod 2^5 = 1011

例 若X = -1011，则5位字长的$X_{补}$=10101

8位字长的$X_{补}$=11110101

(2^5 -1011) mod 2^5 = (10000-1011)mod(10000)=10101

(2^8 -1011) mod 2^8 = =(10000000-1011)mod(10000000)=11110101

注：补码最高位仍是符号位，但符号位的值是通过定义计算得到的

知识点

补码：00111等值8位字长：0(000)0111

00111原码：00111，00(000)111

10111等值8位字长：1(111)01111

10111原码：11001，　1(000)1001

带符号数的补码扩展原则：是用符号位填充扩展高位

补码表示区间

- 定点小数 $x_0.x_1x_2\cdots x_n$
  - $-1.00\cdots0 \sim +0.11\cdots1$
  - $[-1, 1-2^{-n}]$ 或 $[-1, 1)$
- 定点整数 $x_0x_1x_2\cdots x_n$
  - $-100\cdots0 \sim +011\cdots1$
  - $[-2^n, 2^n-1]$ 或 $[-2^n, 2^n)$
- 非对称区间，左侧多一个数

注：补码相对原码反码少一个负零，多表示一个负一。

负数比正数多一个数

字长8位的补码

表示范围为：-128 ～ +127

$[+127]_{补}=0\ 1111111$；

$[-128]_{补}=1\ 0000000$

※注意

补码比原码和反码多表示1个负值，即–128

数值“0”只有1种补码形式：

$[+0]_{补}=[-0]_{补}=0\ 0000000$

二进制代码	无符号数值	原码值	反码值	补码值
0000 0000	0	+0	+0	+0
0000 0001	1	+1	+1	+1
⋮	⋮	⋮	⋮	⋮
0111 1110	126	+126	+126	+126
0111 1111	127	+127	+127	+127
1000 0000	128	-0	-127	-128
1000 0001	129	-1	-126	-127
1000 0010	130	-2	-125	-126
⋮	⋮	⋮	⋮	⋮
1111 1101	253	-125	-2	-3
1111 1110	254	-126	-1	-2
1111 1111	255	-127	-0	-1

3、补码编码规则

例 若X=0.1011，则8位字长的 $X_{补}$ = 0.1011000

(2+0.1011000) mod 2 = 0.1011000

$X_{原}$ = 0.1011000

例 若X = -0.1011，则5位字长$X_{补}$ = (2 - 0.1011)mod2=1.0101

写成8位字长，则$X_{补}$ = (2 - 0.1011000) mod 2 = 1.0101000

$X_{原}$ = 1.1011000

$X_{反}$ = 1.0100111

例 若X = 1011，则5位字长的$X_{补}$=01011

$X_{原}$ =01011

8位字长的$X_{补}$=00001011

$X_{原}$=00001011

例 若X = -1011，则5位字长的$X_{补}$=10000-1011=10101

$X_{原}$=11011

$X_{反}$=10100

8位字长的$X_{补}$=10000000-1011=11110101

$X_{原}$=10001011

$X_{反}$=11110100

3、补码规则

※补码的编码规则：

(a)对于正数（字长8位）

$[X]_{补}=[X]_{原}$ （即X≥0时）

(b)对于负数（字长8位）

符号位仍保持为“1”

其余各数值位“按位取反，末位再加1”

$[X]_{补}=[X]_{反}+\ldots\underline{1}$ （即X<0时）

4、原码和补码之间的转换

(1) 已知$[X]_{原}$，求$[X]_{补}$

[例] 已知$[X]_{原}$= 10011010，求$[X]_{补}$

解：$[X]_{原}$= 10011010　原码为负数

↓↓↓↓↓↓↓↓　符号位不变，其余各位变反

11100101

\+ 1　末位加1

$[X]_{补}$=11100110

练习

- X=-0.11111111

  $[X]_{补}$=

- X=-0.00000001

  $[X]_{补}$ =

- X=-000000001

  $[X]_{补}$ =

练习

- X=-0.11111111

$[X]_{补}$ =1.00000000
+0.00000001
=1.00000001

- X=-0.00000001

$[X]_{补}$ =1.11111111

- X=-000000001

$[X]_{补}$ =111111111

(2) 已知$[X]_{补}$，求$[X]_{原}$

$$[[X]_{补}]_{补}=[X]_{原}$$

[例] 已知$[X]_{补}=11101100$，求$[X]_{原}$

解：$[X]_{补}=1\,1101100$　　补码为负数

↓↓↓↓↓↓↓　　符号位不变，其余各位变反

1 0010011

\+ 1　　末位加1

$[X]_{原}=1\,0010100$

练习

$[X]_{补} = 1.00000001$

$[X]_{补} = 1.11111111$

$[X]_{补} = 111111111$

求$X_{原}$？？

练习

$[X]_{补}$ =1.00000001

- X=-0.11111111

$[X]_{补}$ =1.11111111

- X=-0.00000001

$[X]_{补}$ =111111111

X=-000000001

(3) 求补(变补)，即已知$[X]_{补}$，求$[-X]_{补}$

$[X]_{补}$的代码连同符号位一起变反，末位再加1，即得到$[-X]_{补}$

[例] 已知$[X]_{补}=01010110$，求$[-X]_{补}$

解：$[X]_{补}=0\,1010110$　　不区分正负数

↓↓↓↓↓↓↓↓　　连同符号位一起变反

$1\,0101001$

$+\qquad\qquad 1$　　末位加1

$[-X]_{补}=1\,0101010$

(4) 求补(变补)，双符号位补码

$$[X]_{补}=\begin{cases} X & 0\le X<2^{n} \\ 2^{n+2}+X & -2^{n}\le X<0 \end{cases} \quad \text{mod } 2^{n+2}$$

例：00001010110

11101010001

$$[X]_{补}=\begin{cases} X & 0\le X<1 \\ 4+X & -1\le X<0 \end{cases} \quad \text{mod } 4$$

例：00.01010110

11.01010001

符号位01表示正溢出，10表示负溢出，最高位表示正确符号位

补码特性

- 零有唯一的表示方式
- 定点小数补码相对原码少一个负零，多表示一个-1，其代码序列为1.0000….。
- 定点整数负数比正数多一个数-2^n,其代码序列为100000……。
- 减法可以变成加法（反相器），运算电路统一
- 符号位可以直接参与运算

注：定点纯小数的-1和定点纯整数的-2^n的真值只能用定义求，因为其没有原码。

练习

1 下列关于补码机器数的描述中错误的是（　　）（单选）

A.数的符号用0和1表示

B.数值“0”的表示唯一

C.加法和减法按模进行运算

D.不可能出现一个数的补码与其原码相同

练习

1 下列关于补码机器数的描述中错误的是（　　）　（单选）

A.数的符号用0和1表示

B.数值“0”的表示唯一

C.加法和减法按模进行运算

D.不可能出现一个数的补码与其原码相同

5、移(增)码

移码通常用于表示浮点数的阶码。

阶码一般为整数，故移码通常只用于表示整数

对定点整数x，它的移码是：

$$[x]_{移}=2^{n-1}+x，\text{其中}-2^{n-1}<x<2^{n-1}$$

这里的n为$X_{原}$位数

上述规则等价于将x正向平移或者增加2^{n-1}，因此称之为移码或增码。

[例] 阶码的为6位，X表示其真值

$X_{移}=2^5+X$ （$-2^5<X<2^5$）

[例] 当正数X＝+10101时, $X_{移}=2^5+X=110101$

当负数X＝-10101时, $X_{移}=2^5+X$

$=2^5-10101$

$=001011$

移码表示范围与补码一致，0也只有1个移码。

正数：将原码符号位变反，即得到移码。

负数：将原码连同符号位一起变反，末位再加1，即得到移码(与变补等效)。

补码和移码：符号相反、数值位相同

移码表示法特点

- 移码符号位为0时X为负，为1时X为正
- 除符号位相反之外，移码的其余各位与补码相同
- 让X从 - 128逐渐增至 + 127，相应地$X_{移}$从00…00增至11…11，呈递增状

真值X (十进制)	真值X (二进制)	$X_{移}$	$X_{补}$
- 128	- 10000000	00000000	10000000
- 127	- 01111111	00000001	10000001
…			
- 1	- 00000001	01111111	11111111
0	00000000	10000000	00000000
+ 1	00000001	10000001	00000001
…			
+ 127	01111111	11111111	01111111

机器码小结-1

数值数据表示法---定点数的表示

第一个知识点小结



原码

- 直观易懂第一位为符号位，其他为数值位；
- 正零负零两个零，加、减运算方式不统一。
- 符号相异加法不能直接运算。



补码

- 补唯一的零，定点小数补码相对原码少一个负零，多一个负一；定点整数补码相对原码少一个负零，多一个负数。
- 符号位可以直接参与运算，减法可以变成加法



移码

- 移码与补码的符号位相异，数据位相同，移码用于浮点数阶码。
- 移码保持数据原有大小顺序，便于进行比较操作。

机器码小结-2

- 4种编码方式-求法
  - 原码　最高位符号位，0为正，1为负，数值位不变
  - 反码　正同原，负：符号位同原，其余取反
  - 补码　正同原，负：符号位同原，其余取反，末位+1
  - 移码　补码符号位取反

机器码小结-3

- n+1位数，数据位n位

	定点整数		定点小数	
原码反码	$[1-2^n, 2^n-1]$	$(-2^n, 2^n)$	$[-2^{-n}-1, 1-2^{-n}]$	$(-1, 1)$
补码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	$[-1, 1-2^{-n}]$	$[-1, 1)$
移码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	小数无移码	

机器码小结-4

- 4种编码方式-用法
  - 原码　用来表示浮点（实）数的尾数
  - 反码　已不用于表示数值数据
  - 补码　50年代开始成为整数标准
  - 移码　用于浮点数阶码

练习

- 1 0的原码可以用下列哪一个代码表示（）。
  A 11111111　B 10000000
  C 01111111　D 1100000
- 2 9位原码能表示的数据个数是（）
  A 10　B 9　C 511　D 512
- 3 计算机系统中采用补码运算的目的是为了（）
  A 与手工运算的方式保持一致　B 提高运算速度
  C 简化计算机的设计　　D 提高运算精度

练习

- 1 0的原码可以用下列哪一个代码表示（B）。
  A 11111111　B 1000000
  C 01111111　D 1100000
- 2 9位原码能表示的数据个数是（C ）
  A 10　B 9　C 511　D 512
- 3 计算机系统中采用补码运算的目的是为了（C）
  A 与手工运算的方式保持一致　B 提高运算速度
  C 简化计算机的设计　D 提高运算精度

练习

- 4 设x=0.1011,求 x的原码、反码和补码。
- 5 设x=-0.1011,求 x的原码、反码和补码。
- 6 将下列数转换成真值

$[x]_{原}$=1.1010 x=

$[x]_{反}$= 1.1100 x=

$[x]_{补}$ = 0.1101 x=

$[x]_{补}$= 1.1101 x=

练习

- 1 设x=0.1011,求 x的原码、反码和补码。
  $[x]_{原}$= 0.1011； $[x]_{反}$ =0.1011； $[x]_{补}$ =0.1011；
- 2 设x=-0.1011,求 x的原码、反码和补码。
  $[x]_{原}$= 1.1011； $[x]_{反}$ =1.0100； $[x]_{补}$ =1.0101；
- 3 将下列数转换成真值

$[x]_{原}$=1.1010 x= -0.1010

$[x]_{反}$= 1.1100 x= -0.0011

$[x]_{补}$ = 0.1101 x= 0.1101

$[x]_{补}$= 1.1101 x=-0.0011

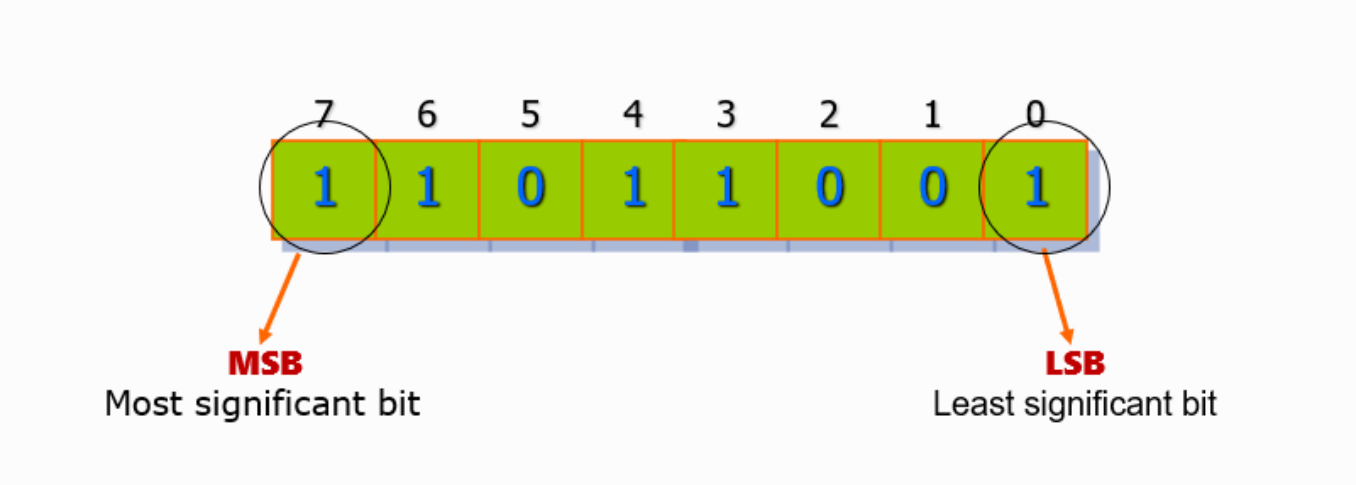
2.1.3 定点数与浮点数

1、定点数的表示

数的小数点固定在同一位置不变。

术语

- 计算机利用寄存器存储数据
- 寄存器中每个位称比特bit (Binary DigiT），每个字节8bit。
- 最高有效位 (**MSB**) 最低有效位 (**LSB**)



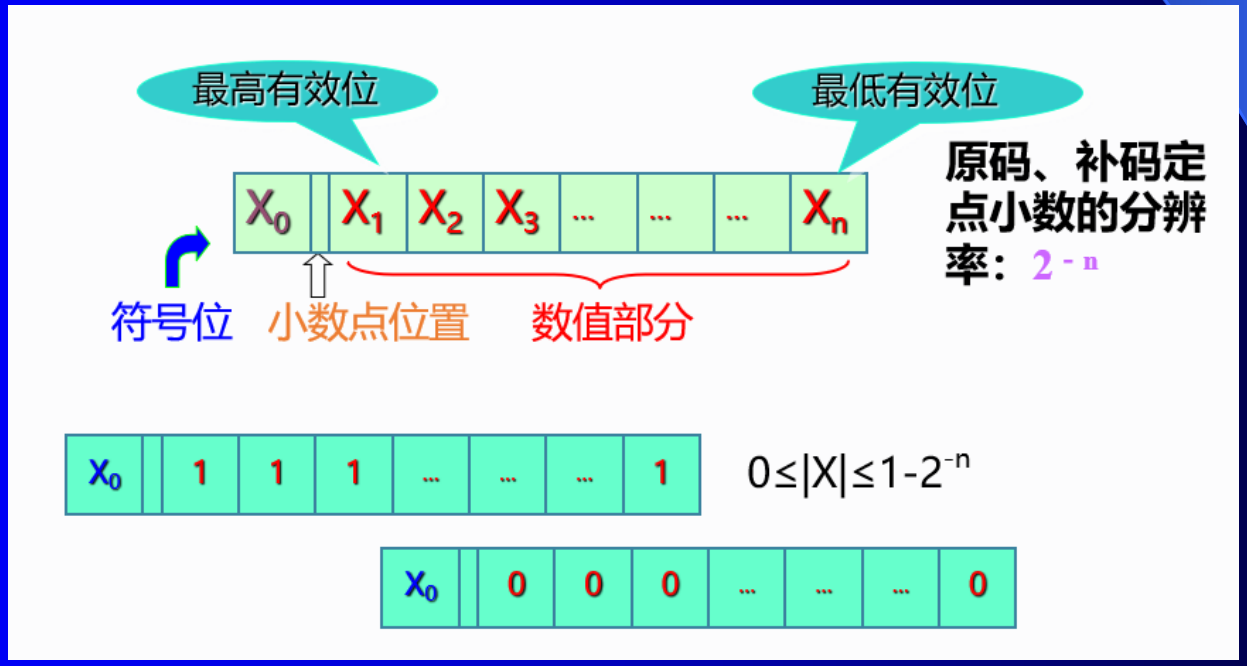
2.1.3 定点数与浮点数

1、定点数的表示

数的小数点固定在同一位置不变。

①带符号的定点小数

约定所有数的小数点的位置，固定在符号位之后。

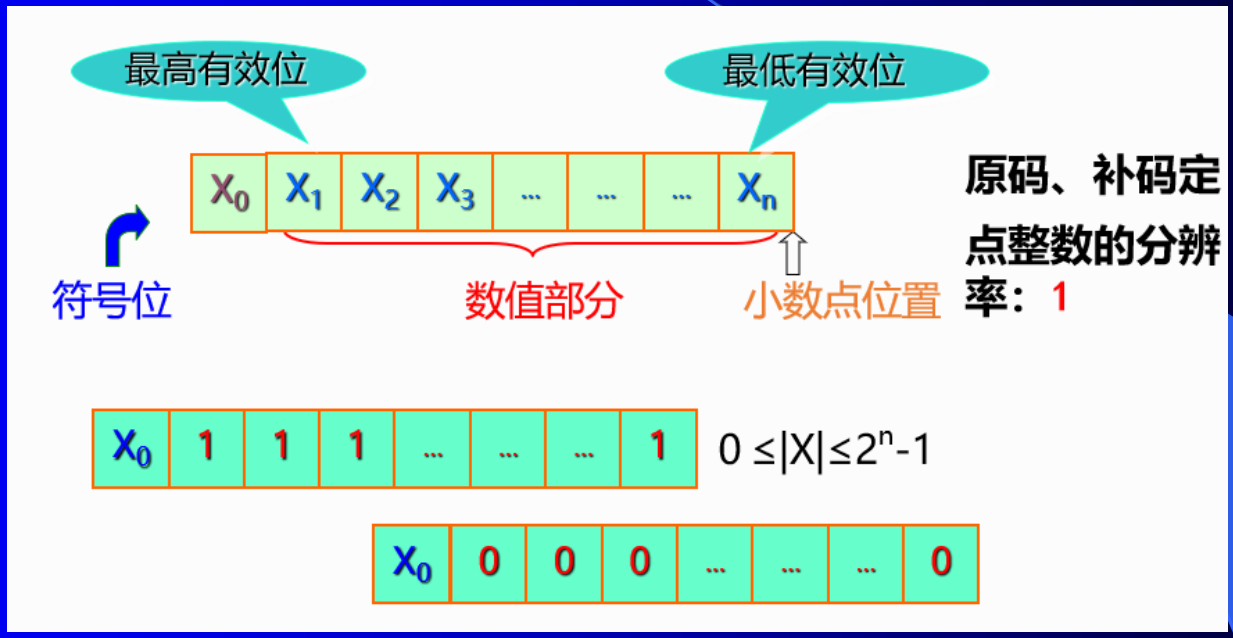


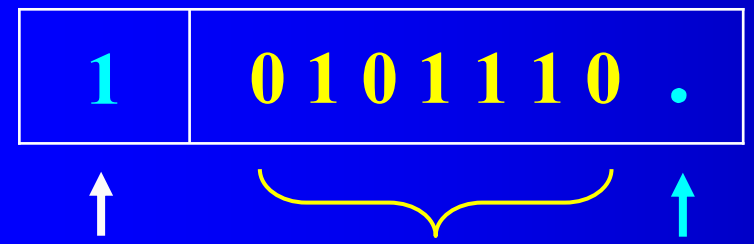
例：



②带符号的定点整数

小数点的位置固定在最低数值位之后，





带符号的定点数表示范围

定点小数表示范围-以原码为例

典型值	真值	代码序列
最小负数		
最大负数		
最小非零正数		
最大正数		

小数：$[-(1-2^{-n}), 1-2^{-n}]$　　或　　$(-1,1)$

带符号的定点数表示范围

定点小数表示范围-以原码为例

典型值	真值	代码序列
最小负数	$-(1-2^{-n})$	1.1…11
最大负数	-2^{-n}	1.0…01
最小非零正数	2^{-n}	0.0…01
最大正数	$1-2^{-n}$	0.1…11

小数：$[-(1-2^{-n}), 1-2^{-n}]$　　或　　$(-1,1)$

带符号的定点数表示范围

定点整数表示范围-以补码为例

典型值	真值	代码序列
最小负数		
最大负数		
最小非零正数		
最大正数		

定点整数：$[-2^n, 2^n-1]$ 或 $[-2^n, 2^n)$

带符号的定点数表示范围

定点整数表示范围-以补码为例

典型值	真值	代码序列
最小负数	-2^n	10…00
最大负数	-1	11…11
最小非零正数	1	00…01
最大正数	2^n-1	01….11

定点整数：$[-2^n, 2^n-1]$ 或 $[-2^n, 2^n)$

带符号定点数机器码表示范围

- n+1位定点数，数据位n位

	定点整数		定点小数	
原码反码	$[1-2^n, 2^n-1]$	$(-2^n, 2^n)$	$[2^{-n}-1, 1-2^{-n}]$	$(-1, 1)$
补码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	$[-1, 1-2^{-n}]$	$[-1, 1)$
移码	$[-2^n, 2^n-1]$	$[-2^n, 2^n)$	小数无移码	

③无符号定点整数

无符号定点整数：略去符号位的正整数称为无符号整数

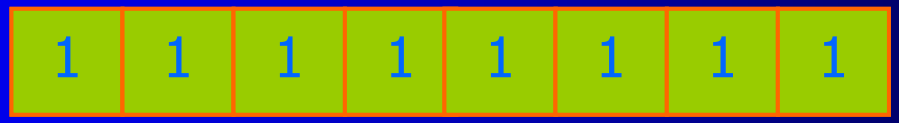
小数点的位置固定在最低数值位之后

若代码序列为$X_nX_{n-1}\cdots X_1X_0$，共n+1位，则有：

典型值	真值	代码序列
最大正数	$2^{n+1}-1$	11…1111
最小非零正数	1	00….001

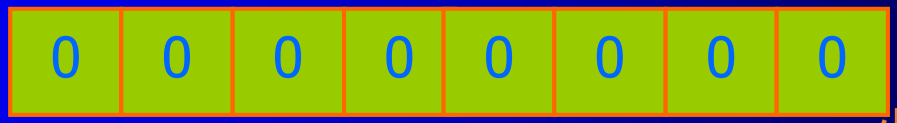
表示范围为：

$0 \sim (2^{n+1}-1)$



小数点位置

分辨率为：1



小数点位置

无符号定点整数特点

- 非负整数；
- 所有的位数都可以用来表示数值的大小。
- 通常用于地址码的表示

※字长8位的定点数的表示范围

无符号数： 00000000 ～11111111

0 255

定点整数： $11111111_{原}$～$01111111_{原}$

-127 127

$10000000_{补}$～$01111111_{补}$

-128 127

定点小数： $1.1111111_{原}$～$0.1111111_{原}$

$-(1-2^{-7})$ $(1-2^{-7})$

$1.0000000_{补}$～$0.1111111_{补}$

-1 $(1-2^{-7})$

注：定点数的表示范围有限，若运算结果超出表示范围，大于正整数，称为正溢。小于定点数的最小值，负溢。

C语言中的定点数

- 无符号整数
  - **unsigned char**
  - **unsigned short**
  - **unsigned int**
  - 一般用于地址运算，编号表示
- 有符号整数
  - **char short int long**
  - 采用补码表示
- 无符号整数的最大值大于位数相同的带符号整数的最大值
  - **8位无符号整数最大是255（1111 1111）**
  - **8位带符号整数最大为127（0111 1111）**

C语言中的整数小结

- 整数补码存储，表示，运算
- C编译器根据常量值的大小自动匹配数据类型
  - -2147483648被当成无符号数 (C90)
  - #define INT32_MIN (-2147483647 - 1)
- 有符号和无符号共存时无符号优先
  - -1 > 0U
- 整数变量初始化
  - 先按约定处理常量 转换成机器码
  - 然后根据变量长度进行强制转换，位数超出的直接截去

定点表示法-讨论

（1）如果某个数既有小数又有整数，要将它规范为某种定点数， 就需要在程序中设置比例因子。

（2）定点数的表示范围有限。如果运算结果超出表示范围，称为 溢出。计算机硬件应具有溢出判断功能。

（3）定点数比较简单，实现定点运算的硬件成本比较低。但在有限位数的定点数中，表示范围与精度这两项指标不能兼顾，选取比例因子比较麻烦。

2、浮点数的表示

数值数据表示法----浮点数的表示

1 浮点数据表示

浮点数的特点：把数的范围和精度分别表示的一种数据表示方法

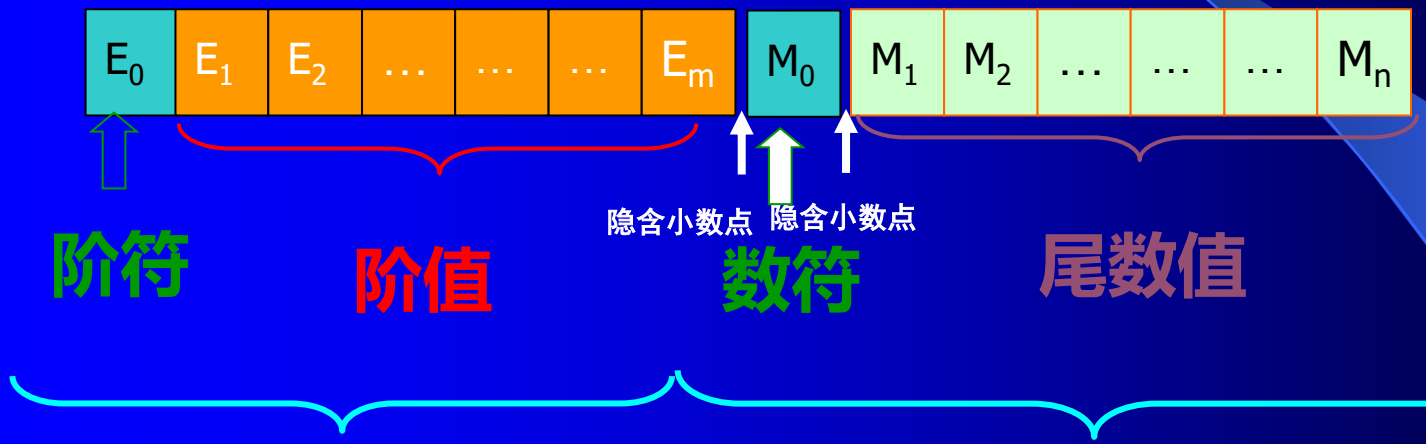
浮点数的使用场合：当数的表示范围超出了定点数能表示的范围时

类型	字节数	有效数字	取值范围（绝对值）
float	4	6	0以及1.2×10^{-38}~3.4×10^{38}
double	8	15	0以及2.3×10^{-308}~1.7×10^{308}
long double	8	15	0以及2.3×10^{-308}~1.7×10^{308}
	16	19	0以及3.4×10^{-4932}~1.1×10^{4932}

2、浮点数的表示原理

①浮点表示中，小数点的位置可按需浮动

格式模型：



（带符号的定点整数）（带符号的定点小数）

②引入浮点数表示的意义

[例]某字长为8位的原码二进制数

定点数：1111111 ~ 0111111

整数：-127　　127　　精度：1

小数：$-(1-2^{-7})$　　$1-2^{-7}$　　精度：2^{-7}

浮点数：5位阶码+3位尾数

01111111 ~ 01111011

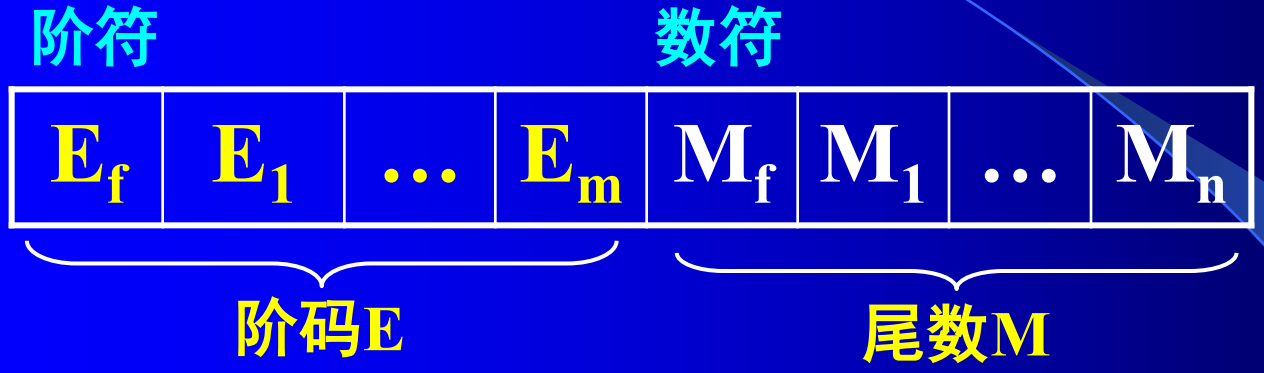
$2^{15}\times(-0.75) \sim 2^{15}\times 0.75$

精度：11111001=$2^{-15}\times 0.25$

相同字长时，浮点数的表示范围更大、精度更高！

③浮点数的机器(存储)格式

浮点数真值： $N = \pm R^{E} \times M$



R：阶码的底数，隐含约定为2。

E：阶码，定点整数，补码或移码表示，其位数决定了数值的范围；

M：尾数，为定点小数，原码或补码表示，其位数决定着数的精度；数符表示数的正负。

④尾数M的规格化表示

规格化的目的 → 使浮点数的表示代码“唯一”

$128=\underline{0.128}\times10^{3}=\underline{1.28}\times10^{2}=\underline{12.8}\times10^{1}$，…

科学计数法约定：$1\le|M|<10$

则规范形式：$128=1.28\times10^{2}$（唯一）

$3.2=1.6\times2^{1}=0.8\times2^{2}=0.4\times2^{3}$，…

可以表示成任意多个代码形式 → 计算不便

约定尾数M的值域，使数的表示是唯一的、确定的

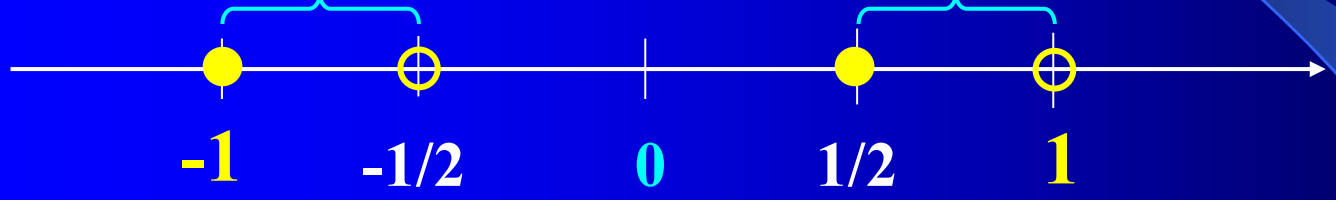
→尾数M的“规格”化

① 浮点数用原码表示时

$1/2 \le |M| < 1$

② 浮点数用补码表示时

$-1 \le M < -1/2$ 或 $1/2 \le M < 1$



求-1/2 的补码？？

对于原码： 规格化以后尾数的最高有效位为“1”

$M_{原}$=0.1000，1.1010 ~~$M_{原}$=0.0010，1.0110~~ 非规格化

对于补码：

正数，规格化后最高数值位为“1” 如0.1010，0.1110

0.625 0.875

负数，规格化后最高数值位为“0” 如1.0010，1.0000

-0.875 -1

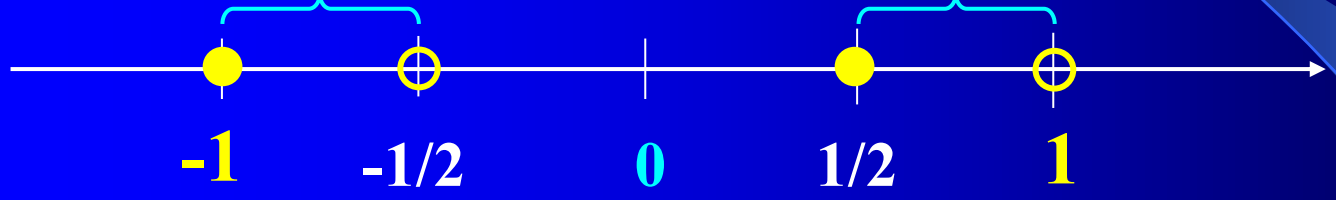
① 浮点数用原码表示时

$1/2 \le |M| < 1$

② 浮点数用补码表示时

$-1 \le M < -1/2$ 或 $1/2 \le M < 1$

机器实际判断补码是否为规格化数时是根据符号位与有效位最高位不同时才是规格化数，-1/2的补码1.10000，-1尾数1.0000



对于原码： 规格化以后尾数的最高有效位为“1”

$M_{原}$=0.1000，1.1010 ~~$M_{原}$=0.0010，1.0110~~ 非规格化

对于补码：

正数，规格化后最高数值位为“1” 如0.1010，0.1110

0.625 0.875

负数，规格化后最高数值位为“0” 如1.0010，1.0000

-0.875 -1

尾数规格化特征：

对于原码表示的尾数，规格化的特征是：$M_1 = 1$；

对于补码表示的尾数，规格化的特征是：尾数最高数位和符号位不同。

假设浮点数的R＝2，其尾数如下，判别哪些是规格化尾数

A补＝1.0110001　　B补＝1.0000000　　C原＝1.1000000

D原＝1.0110011　　E补＝0.0111111　　F补＝0.1000000

(1) 浮点表示法

假设浮点数的R＝2，其尾数如下，判别哪些是规格化尾数

A补＝1.0110001　　B补＝1.0000000　　C原＝1.1000000
D原＝1.0110011　　E补＝0.0111111　　F补＝0.1000000

根据规格化尾数的特征可判断出A补、B补、C原、F补是规格化的尾数

[例1]阶符1位、阶码m位，数符1位、尾数n位，如果表示成规格化补码，分析各真值对应的M、E取值情况。

$$N = M \times 2^E = F(M, E) \begin{cases} -1 \le M < -1/2 \text{或} 1/2 \le M < 1 \\ -2^m \le E \le 2^m - 1 \end{cases}$$

最小浮点数	E→ 为最大正数： 2^m-1
	M→ 为最小负数： -1
最大浮点数	E→ 为最大正数： 2^m-1
	M→ 为最大正数： $1-2^{-n}$
最小浮点正数	E→ 为最小负数： -2^m
	M→ 为最小正数： 2^{-1}
最大浮点负数	E→ 为最小负数： -2^m
	M→ 为最大负数： $-(1/2+2^{-n})$

[例2]某个用规格化补码表示的浮点数，其中阶码6位（含1位阶符），尾数10位（含1位数符）。

表示范围：011111 100…00 ~ 011111 011…11

$2^{31} \times (-1)$ $2^{31} \times (1-2^{-9})$

绝对精度（即最小的非0正数）：

100000 010…00 $=2^{-32} \times 0.5=2^{-33}$

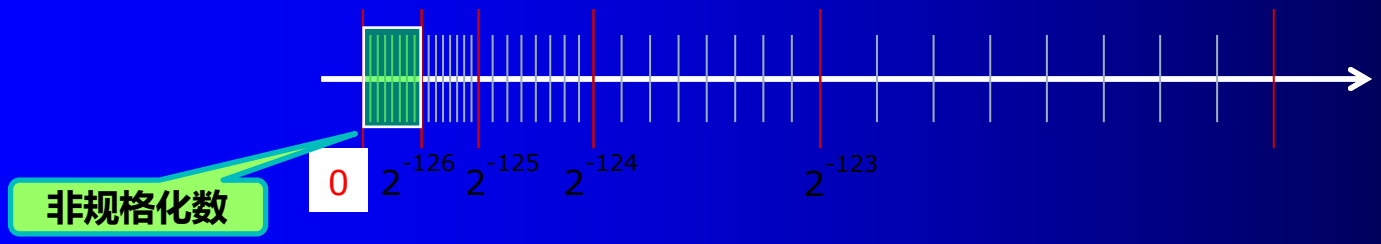
相对精度（只与尾数的数值位数有关）：

xxxxxx 000…01 $=2^{-9}$

将浮点数的精度分为相对精度与绝对精度。相对精度是尾数本身的分辨率，取决于尾数的位数。绝对精度 = R^E × 相对精度。

浮点数的表示范围与精度

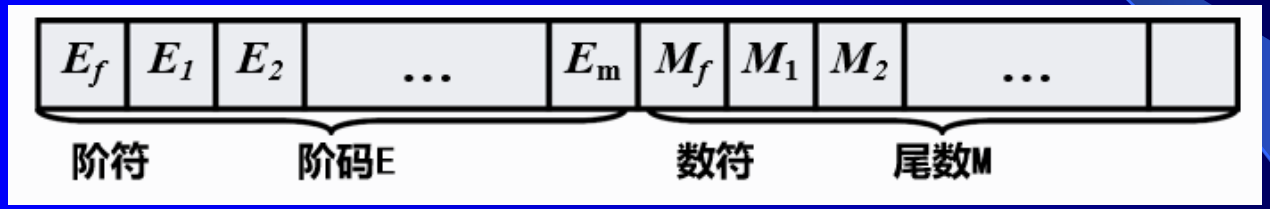
- 阶码越长，表示范围越大
- 阶码相同，尾数越长，数据精度越高
- 浮点数扩大了数值表示的范围
- 绝对值越大，浮点数分布越稀疏，浮点数是离散空间



① 真值与浮点数之间的转化

某十六进制浮点数$A3680000_{16}$，将其表示成补码，字长32位，阶码8位（含1位阶符），尾数24位（含1位数符），求该浮点数十进制的真值。

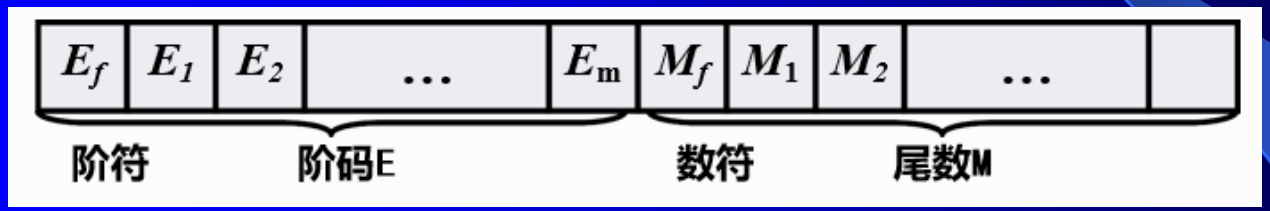
$N=2^E \times M$



① 真值与浮点数之间的转化

某十六进制浮点数$A3680000_{16}$，将其表示成补码，字长32位，阶码8位（含1位阶符），尾数24位（含1位数符），求该浮点数十进制的真值。

$N=2^E \times M$



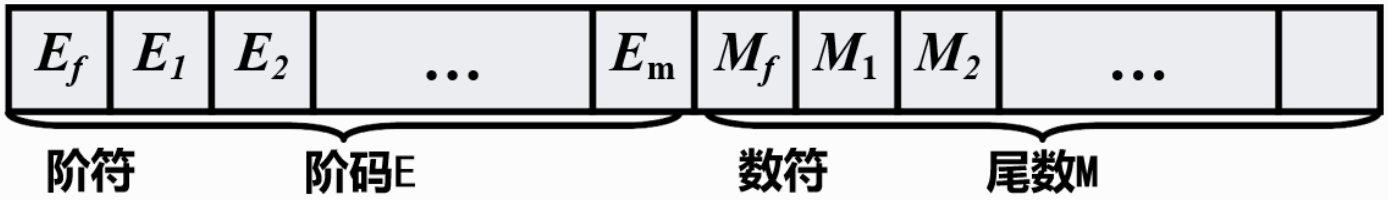
$(A3680000)_{16} = (1010\ 0011, 0110\ 1000\ 000\cdots0)_2$

$E = -(1011101)_2 = -(93)_{10}$

$M = (0.11010\cdots0)_2 = (0.8125)_{10}$

$N = 2^{-93} \times 0.8125$

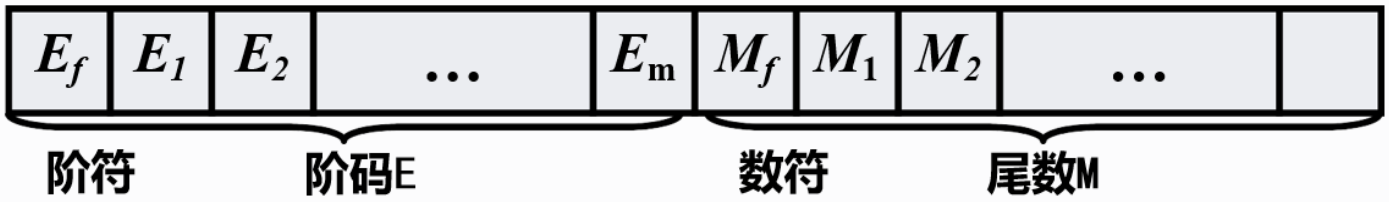
① 真值与浮点数之间的转化



例　按上述浮点格式将 - $(1011.11010\cdots0)_2$写成浮点数代码$N=2^{\pm e}\times(\pm m)$

字长32位，阶码8位（含1位阶符），尾数24位（含1位数符）

① 真值与浮点数之间的转化



例　按上述浮点格式将 - (1011.11010…0)$_2$写成浮点数代码$N=2^{\pm e}\times(\pm m)$

$N = -(1011.11010\cdots0)_2$

$= -(0.101111010\cdots0)_2\times2^4$

$E = (4)_{10} = (00000100)_2$

$E_{补} = 00000100$

$M_{原} = -(0.101111010\cdots0)_2$

$M_{补} = (1.010000110\cdots0)_2$

浮点数代码为 $(00000100, 1010000110\cdots0)_2 = (04A18000)_{16}$

浮点数溢出问题

2 数的浮点表示法 (5) 溢出问题

- 定点数判断溢出的办法是对数值本身进行判断
- 浮点数是对规格化后的阶码进行判断
  - 当一个浮点数阶码大于机器的最大阶码，称为上溢；
    机器产生上溢时，不能再继续运算，一般要进行中断处理。
  - 而小于最小阶码时，称为下溢。
    出现下溢时，一般规定把浮点数各位强迫为零(当做零处理)，机器仍可继续进行运算。

思考：定点数与浮点数的区别？

- （1） 数值的表示范围： 如果定点数和浮点数的位数相同，那么浮点表示法所能表示的数值范围将远远大于定点表示法。浮点数的阶码部分的位数越 多，可表示的数值范围越大。
- （2） 精度： 精度是指一个数所能包含的有效数值位的位数。一般来说，机器字长越长，所能表示的数的有效位数就越多，精度就越高。
- （3） 数的运算： 浮点数包括阶码和尾数两部分，运算时需要对尾数和阶码分别计算。而且运算结果都要规格化。
- （4） 溢出处理： 在定点数运算时，当运算结果超出数的表示范围时，就发生溢出；在浮点运算时，运算结果超出尾数的表示范围不一定溢出， 只有当阶码超出所能表示的范围时，才发生溢出。

练习

- 1 长度相同但格式不同的2种浮点数，假设前者阶码长，尾数短。后者阶码短，尾数长，其他规定相同，则他们表示的数的范围和精度为（）

  A 两者相同

  B 前者表示数的范围大精度低

  C后者表示数的范围大精度低

  D 前者表示数的范围大精度高

2 在浮点原码运算时，判定为规格化数的条件是（　）

A 阶码的符号位与尾数符号位不同

B 尾数的符号位与最高数值位相同

C 尾数的符号位与最高数值位不同

D 尾数的最高数值位为1

练习

- 1 长度相同但格式不同的2种浮点数，假设前者阶码长，尾数短。后者阶码短，尾数长，其他规定相同，则他们表示的数的范围和精度为（B）

  A 两者相同

  B 前者表示数的范围大精度低

  C后者表示数的范围大精度低

  D 前者表示数的范围大精度高

2 在浮点原码运算时，判定为规格化数的条件是（D）

A 阶码的符号位与尾数符号位不同

B 尾数的符号位与最高数值位相同

C 尾数的符号位与最高数值位不同

D 尾数的最高数值位为1

3、※IEEE754格式的浮点数

- 80年代初，各机器内部浮点数表示还没有统一
- 相互不兼容，机器之间传送数据时很麻烦
- 1970年代后期, IEEE着手制定浮点数标准
- 1985年完成浮点数标准IEEE 754的制定，沿用至今

This standard was primarily the work of one person, UC Berkeley math professor William Kahan.

1989
ACM Turing
Award Winner!

www.cs.berkeley.edu/~wkahan/ieee754status/754story.html



Prof. William Kahan

浮点数标准 IEEE754



- 构成：符号位S，阶码E，尾数M，
- $N = (-1)^S \times M \times 2^{E-x}$

✓ 符号s（sign）：s决定这数是负数（s=1）还是正数（s=0），对于数值0的符号位解释作为特殊情况处理

✓ 尾数M（significand）：M是一个二进制小数

✓ 阶码E（exponent）：采用偏移值，是对浮点数加权，这个权重是2的E-x次幂，当为单精度是X=127，为双精度是X=1023，即将浮点数的阶码值变成非负整数，便于比较和排序

有32位浮点数（单精度）和64位浮点数（双精度）

① 32位短浮点数：

$$(-1)^{S} \times 1.M \times 2^{E-127}$$

31	30～23	22～0
S（1bit）	E （8bit）	M （23bit）

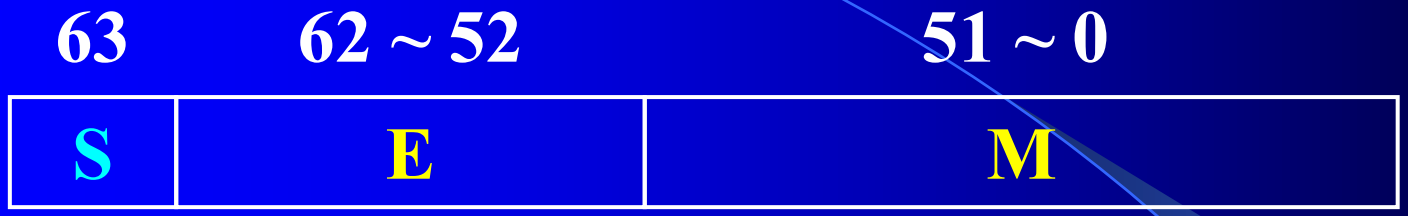
在上述的表示格式中：

- S=浮点数的符号位，0表示正数，1表示负数；
- E=阶码，8位，采用移码表示，阶符隐含；
- M=尾数，23位，原码，纯小数表示，且真值=1+M；

因规格化数最高数字位恒为1，该位为隐藏位，可节约尾数数据位

- 阶码E采用移码形式，但只偏移2^7-1（不是2^7）。

② 64位长浮点数：



在上述表示格式中：

- S=浮点数的符号位，0表示正数，1表示负数；
- E=阶码，11位，采用移码方式，阶符隐含；
- M=尾数，52位，用纯小数表示；
- 对阶码E编码时，只偏移$2^{10}-1$（标准移码偏移2^{10}）。

※补充说明：

（1）为了确保浮点数表示的唯一性，约定 $0 \le M < 1$；

（2）E为全0且M非全0：非规范浮点数（E偏移126），则：$F_{真}=(-1)^S \times M \times 2^{E-126}$

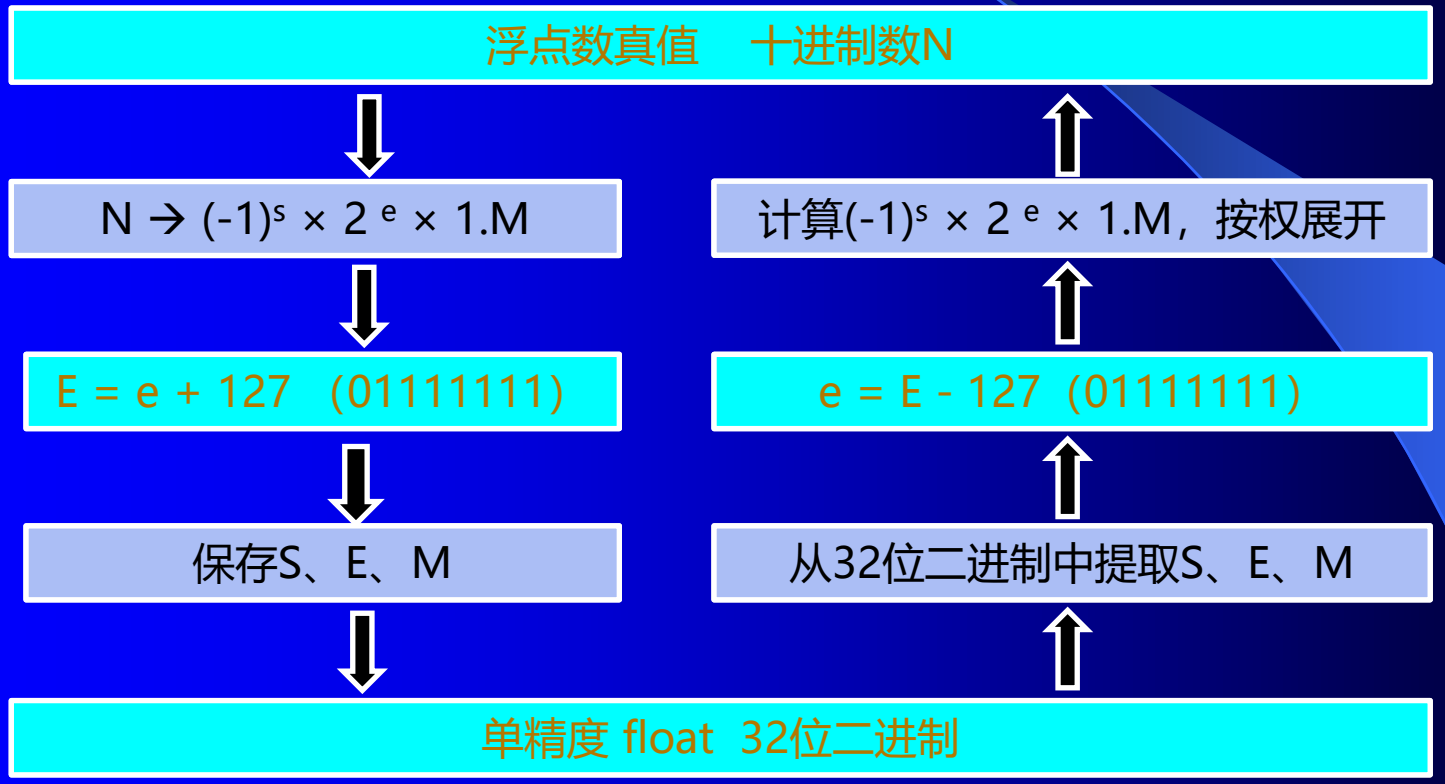
（3）E为全0且M为全0：表示浮点数 0

（4）$1 \le E \le 254$：数是规范浮点数（E偏移127），则：$F_{真}=(-1)^S \times (1+M) \times 2^{E-127}$

（5）E为全1（255）：M为全0，则$F_{真}=\pm\infty$

（6）E为全1（255）：M非全0时，代码无效(NaN)；

float型与真值之间的变换流程



[例] 将十进制数20.59375转换成IEEE754的32位标准浮点数的2进制格式，并写出相应的16进制数。

[解答] 首先分别将整数和小数部分转换成二进制：

20.59375＝10100.10011

非规格化形式转换成规格化形式1.M

10100.10011=1.010010011×2^4

提取S,E,M

符号位S=0，阶码E=4＋127=131，尾数M=010010011

最后得到32位浮点数的二进制代码：

0100 0001 1010 0100 1100 0000 0000 0000

＝$(41A4C000)_{16}$

- **例：将短浮点数C1C90000转换成十进制数**

1 十六进制→二进制形式，并分离出符号位、阶码和尾数。

C1C90000H=1;10000011;10010010000000000000000

2 计算出阶码E真值

e=10000011-01111111=100=4$_{(10)}$

3 以规格化二进制数形式写出此数　1.1001001×2^4

4 写成非规格化二进制数形式　11001.001

5 转换成十进制数，并加上符号位。

$(11001.001)_2=(25.125)_{10}$

所以，该浮点数=-25.125

练习

1 将短浮点数36.625转换成IEEE754格式

解： 将36.625转换成二进制形式 36.625=100100.101

2 若浮点数N 的二进制存储格式为 $(41360000)_{16}$，求此IEEE 754格式32位浮点数的十进制值。

解：将十六进制数展开后，可得二进制数格式为

0 100 0001 0011 0110 0000 0000 0000 0000

练习

将短浮点数36.625转换成IEEE754格式

1将36.625转换成二进制形式 36.625=100100.101

2非规格化形式转换成规格化形式1.M

$100100.101=1.00100101\times2^{5}=1.00100101\times2^{101}$

3 提取S,E,M

符号位 S=0

阶码 E=5+127=101+01111111=10000100

尾数 M=00100101000000000000000

4 32位： 0;100 0010 0;001 0010 1000 0000 0000 0000
表示为十六进制的代码：42128000H。

2 若浮点数N 的二进制存储格式为（$41360000)_{16}$，求此IEEE 754格式32位浮点数的十进制值。

解：将十六进制数展开后，可得二进制数格式为

$$0 \quad 100\ 0001\ 0011\ 0110\ 0000\ 0000\ 0000\ 0000$$

从 32 位二进制串中分离出 S,E,M：

$$S=0,\quad E=10000010,\quad M=011\ 011$$

进一步计算可得：

$$e=E-01111111=10000010-01111111=00000011=(3)_{10}$$

实际尾数 $1.M=1.011\ 011=1.011011$

于是有：

$$N=(-1)^S\times 1.M\times 2^e=(1.011011)\times 2^3=+1011.011=(11.375)_{10}$$